

Micro-Expression Recognition on CASME II — Results Explainer

A concise walkthrough of what we ran, what we found, and why — for discussion with my supervisor.

1. What the experiment is

We are recognising **micro-expressions** (involuntary facial movements lasting $< 1/5$ second) on the **CASME II** dataset. Instead of proposing one architecture and claiming it works, we ran an **ablation study**: we take four components and switch each one on or off in every combination, so we can see *which parts actually help*.

The four components (toggles):

Toggle	Component	What it does
EVM	Eulerian Video Magnification	amplifies tiny, near-invisible motion before feature extraction (a data pre-step)
SimAM	parameter-free spatial attention	re-weights the important regions of the CNN's feature maps (adds 0 parameters)
CNN	three-stream 3-D CNN	extracts spatial + short-range motion features — the "backbone"
Transformer	SLSTT temporal encoder	models long-range relationships across the 32 frames

Four on/off switches $\rightarrow 2^4 = 16$ combinations; 4 are invalid (SimAM needs a CNN to attend to), leaving **12 configurations** that we train and test.

Input: every clip is turned into a **3-channel motion tensor** [3, 32, 224, 224] — horizontal optical flow, vertical optical flow, and optical strain (deformation). We feed **motion, not raw pixels**, because a micro-expression *is* motion, and this also stops the model from recognising the person's face instead of the expression.

The data: CASME II has 255 micro-expression clips. After grouping to 3 classes and dropping the incoherent "Others" bucket, we use **156 clips from 25 subjects**:

Class	Clips	Made from raw emotions
Negative	99	disgust (63) + repression (27) + sadness (7) + fear (2)
Positive	32	happiness
Surprise	25	surprise

Why group 7 emotions into 3? Some raw classes are unusable on their own — **fear has only 2 clips, sadness only 7**. You cannot train or fairly test a 2-clip class. Also, negative emotions (disgust/fear/sadness/repression) share overlapping facial signatures that even human coders confuse. Grouping by valence (Negative / Positive / Surprise) is the standard MEGC challenge protocol and makes our numbers comparable to published work.

2. Why we report Macro-F1, not accuracy

This is the most important thing to explain, because our accuracy numbers look modest at first glance.

The classes are very imbalanced ($\approx 63\%$ **Negative**, 20% Positive, 16% Surprise). A lazy model that **always predicts "Negative"** and learns nothing scores:

"Always predict Negative"	Accuracy	Macro-F1
LOSO	0.662	0.266
Hold-out	0.750	0.286

So a useless model gets **66–75 % accuracy**. If we optimised for accuracy, we would be *rewarding* the model for ignoring the two rare classes.

Macro-F1 averages the F1 score of each class *equally*, so ignoring Positive and Surprise is heavily punished. It is the honest metric for imbalanced problems. We therefore:

- report **Macro-F1 as the headline number**,
- keep accuracy only as a reference (and to expose this exact trap),
- also report **per-class precision/recall/F1** and the **confusion matrix** to show *where* errors happen.

One-line version for the professor: "Accuracy is misleading here because 63 % of clips are one class — a do-nothing model already scores 66–75 %. Macro-F1 measures whether we actually recognise the rare classes, so that's our primary metric."

3. The two validation methods (brief)

Both methods are **subject-disjoint** — the same person is *never* in both training and testing. This is essential: if clips of one person appear on both sides, the model can recognise the *face* instead of the *expression* (this is called **identity leakage**), and the score becomes fake.

Hold-out (fast)

- Split the subjects **once**: ~70 % of subjects for training, **30 % of subjects held out for testing** (52 test clips).
- One training run per configuration → cheap and quick.
- Used to iterate on settings. Downside: the result depends on the luck of that one split (higher variance).

LOSO — Leave-One-Subject-Out (rigorous)

- Hold out **one subject**, train on all the others, test on that subject. Repeat for each subject, then pool all predictions.
- This is the gold standard: it tells you how the model does on a **completely new person**, which is what matters in real use.
- Downside: it needs a full training run *per subject* → very slow. So we ran a **pilot: 20 of the 25 subjects** (139 pooled test clips). The full 25-fold run is reserved for the final thesis number.

One-line version: "Hold-out is one quick 70/30 subject split; LOSO trains once per subject and tests on the left-out person. Both keep subjects separate so the model can't cheat by memorising faces. LOSO is the honest generalisation number; hold-out is the fast development check. We ran full hold-out and a 20-of-25 LOSO pilot."

Why 30 % held out (and why not more)? With only ~140 clips, a 10 % test set would be ~15 clips — too few to measure a 3-class score, and a rare class might not appear at all. 30 % (~52 clips) is the smallest test set that keeps all three classes present and measurable. We can't test on 100 % — that would leave nothing to train on.

4. Results

4.1 LOSO (pilot, 20/25 subjects, 139 test clips) — ranked by Macro-F1

Rank	Configuration	EVM	SimAM	CNN	Transf.	Accuracy	Macro-F1
1	attention_base	—	✓	✓	—	0.643	0.379
2	spatial_only	—	—	✓	—	0.627	0.358
3	EVM + CNN	✓	—	✓	—	0.491	0.329
4	EVM + SimAM + CNN	✓	✓	✓	—	0.491	0.324
5	full_no_attention	✓	—	✓	✓	0.419	0.264
6	EVM + Transformer	✓	—	—	✓	0.511	0.256
7	CNN + Transformer	—	—	✓	✓	0.476	0.252
8	temporal_only	—	—	—	✓	0.600	0.251
9	full (no EVM)	—	✓	✓	✓	0.474	0.250
10	proposed (all 4 on)	✓	✓	✓	✓	0.413	0.249
11	pure_base	—	—	—	—	0.295	0.179

Rank	Configuration	EVM	SimAM	CNN	Transf.	Accuracy	Macro-F1
12	motion_amp_base	✓	—	—	—	0.305	0.166
—	do-nothing baseline					0.662	0.266

4.2 Hold-out (30 % subjects, 52 test clips) — ranked by Macro-F1

Rank	Configuration	EVM	SimAM	CNN	Transf.	Accuracy	Macro-F1
1	EVM + CNN	✓	—	✓	—	0.577	0.458
2	EVM + SimAM + CNN	✓	✓	✓	—	0.558	0.448
3	attention_base	—	✓	✓	—	0.481	0.402
4	spatial_only	—	—	✓	—	0.462	0.388
5	CNN + Transformer	—	—	✓	✓	0.442	0.274
6	temporal_only	—	—	—	✓	0.231	0.247
7	full (no EVM)	—	✓	✓	✓	0.308	0.218
8	EVM + Transformer	✓	—	—	✓	0.212	0.210
9	proposed (all 4 on)	✓	✓	✓	✓	0.173	0.183
10	full_no_attention	✓	—	✓	✓	0.192	0.176
11	pure_base	—	—	—	—	0.192	0.111
12	motion_amp_base	✓	—	—	—	0.096	0.129
—	do-nothing baseline					0.750	0.286

5. What the results mean (four takeaways)

1. The 3-D CNN is the essential component. In both tables, the top-4 configurations all have the **CNN on** and the **Transformer off**. Every config *without* the CNN sinks to the bottom (the EVM-only model bottoms out at 0.096 hold-out accuracy). The spatial backbone is doing the real work.

2. The Transformer *hurts* at this data size. Comparing pairs that differ only by the Transformer, turning it **on always lowers Macro-F1**:

Same config, Transformer OFF → ON	Protocol	Macro-F1 change
attention_base → full(no EVM)	LOSO	0.379 → 0.250 (-0.13)
spatial_only → CNN+Transformer	LOSO	0.358 → 0.252 (-0.11)
EVM+SimAM+CNN → proposed	Hold-out	0.448 → 0.183 (-0.27)
EVM+CNN → full_no_attention	Hold-out	0.458 → 0.176 (-0.28)

Reason: a 4-layer, 8-head Transformer has ~100,000 parameters, but we only have ~140 training clips — far too few to train it, so it **overfits**. The training logs confirm this (training loss keeps dropping while validation F1 stays flat).

3. "More components" is not better. Our fully-loaded *proposed* model (all four on) ranks **10th of 12 (LOSO)** and **9th of 12 (hold-out)** — near the bottom. This is an honest, useful negative result: the effective model is **CNN + attention**, and the Transformer/EVM stack drags it down at this scale.

4. EVM depends on the test method. EVM helps under hold-out (top-2 configs use it) but the best LOSO model omits it. EVM amplifies real motion *and* noise — the noise doesn't transfer to unseen subjects, so EVM helps on a fixed split but hurts cross-subject generalisation.

Overall message for the professor: *"The CNN backbone carries the signal. The Transformer is theoretically appealing but overfits on 156 clips, so it lowers performance everywhere — including in our full proposed model. Our best honest scores (Macro-F1 0.38 LOSO / 0.46 hold-out) clearly beat the do-nothing baseline (0.27 / 0.29), meaning the model genuinely learns the rare classes. Next step is full 25-fold LOSO and scaling to a larger composite dataset so the Transformer finally has enough data to help."*

6. The figures (what each one shows)

All plots are in [results_weekend/loso/plots/](#) and [results_weekend/holdout/plots/](#).

Figure	What it shows	Why it matters
accuracy_macro_f1_bar.png	Blue = accuracy, orange = Macro-F1, per config	The blue bar is always taller than orange → visual proof of majority-class bias, and why we lead with Macro-F1
per_class_f1_grouped.png	F1 for Negative / Positive / Surprise per config	Negative is easy for everyone; the ranking is decided on the rare Positive & Surprise classes
confusion_matrices.png	3×4 grid of confusion matrices (all configs)	Good configs show a clear diagonal; overfit/Transformer configs show "column collapse" (predicting one class)
key_configs_confusion_side_by_side.png	Headline configs compared directly	The proposed model's smeared matrix vs the clean diagonal of the lean CNN models — the negative result made visual

7. Key settings (for reference)

- **Loss:** Focal loss ($\gamma=2$) + label smoothing + inverse-frequency class weights + balanced oversampling → three stacked defences against the 63/20/16 imbalance.
- **Optimiser:** AdamW, lr $1e-4$, weight decay $1e-4$, gradient clipping 1.0, mixed precision, CosineAnnealing schedule.
- **Batch size 2:** a hardware limit — one [\[3,32,224,224\]](#) clip is large (~5 GB VRAM for Transformer configs).
- **Best checkpoint selected by Macro-F1** (not accuracy), consistent with our primary metric.
- **Seed 42** for reproducibility.

All numbers come from [results_weekend/{loso,holdout}/summary.csv](#) and each config's [final_results.json](#); class counts from [master_thesis_labels.csv](#); the 30 % hold-out from [gui_settings.json](#).